Tema IV – Pilas

Las estructuras de datos que estudiaremos a partir de este capítulo son la materialización práctica de los conceptos teóricos fundamentales establecidos anteriormente: abstracción, especificación de TADs, análisis de complejidad y gestión de memoria dinámica. La pila (*stack*) representa una de las estructuras de datos lineales más elementales y, simultáneamente, más ubicuas en la computación.

Una pila es una estructura de datos lineal y dinámica que implementa el principio LIFO (*Last In, First Out*), que en español se traduce como "último en entrar, primero en salir". Esta analogía con una pila de objetos físicos (como una pila de platos o una pila de libros) es perfectamente ilustrativa: solo podemos añadir o quitar elementos por la parte superior, y el último elemento añadido será siempre el primero en ser retirado.

La importancia de las pilas trasciende su simplicidad conceptual. Constituyen un componente fundamental en múltiples aspectos de los sistemas computacionales: desde la gestión de llamadas a subprogramas (el stack de ejecución) hasta algoritmos de procesamiento de lenguajes (evaluación de expresiones, análisis sintáctico) y características de interfaz de usuario (historial de navegación, deshacer/rehacer). Este capítulo nos guiará a través de la especificación, implementación y aplicaciones prácticas de esta estructura fundamental.

1. El TAD Pila

Independientemente de cómo se implemente internamente, un Tipo Abstracto de Datos Pila debe proporcionar un conjunto mínimo de operaciones que respeten el contrato LIFO. La especificación en lenguaje natural sería:

- **Nombre:** Pila
- **Valores posibles:** Vacía o conteniendo una secuencia de elementos del tipo base, donde solo el elemento más reciente (la cima) es directamente accesible.
- **Operaciones básicas:**
  - `initialize(p)` o `crear()`: Inicializa una pila vacía.
  - `push(p, x)`: Inserta el elemento `x` en la cima de la pila `p`.
  - `pop(p)`: Elimina el elemento situado en la cima de la pila `p`.
  - `peek(p)` o `top(p)`: Consulta (sin eliminar) el elemento situado en la cima de la pila `p`.
  - `isEmpty(p)`: Determina si la pila `p` no contiene elementos.
- **Uso de otros tipos:** Depende del tipo base de elementos que la pila contendrá (enteros, caracteres, registros, etc.) y de las estructuras auxiliares utilizadas en la implementación (arrays, punteros).

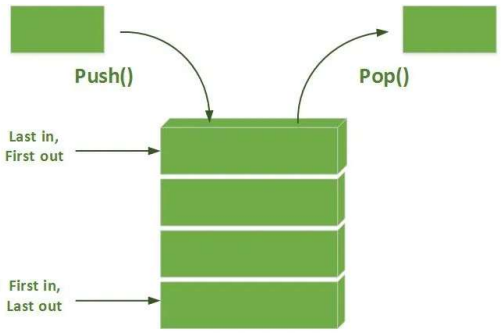

Ilustración 7. Operaciones básicas de un TAD pila

Algunas operaciones incluyen precondiciones que garantizan su correcto funcionamiento:

- **`push(p, x)`**: Añade `x` a la cima de `p`. *Precondición ideal:* La pila no debe estar llena (en implementaciones con capacidad limitada).
- **`pop(p)`**: Elimina el elemento de la cima de `p`. *Precondición obligatoria:* La pila no debe estar vacía (`!isEmpty(p)`). Intentar hacer `pop` en una pila vacía se conoce como underflow y es un error.
- **`peek(p)`**: Devuelve el elemento de la cima sin modificarlo. *Precondición obligatoria:* La pila no debe estar vacía (`!isEmpty(p)`).
- **`isEmpty(p)`**: Devuelve `true` si `p` no contiene elementos, `false` en caso contrario. No tiene precondiciones.

La gestión de estas precondiciones es responsabilidad tanto del implementador (que debe verificar condiciones cuando sea posible) como del usuario (que debe usar el TAD correctamente). En lenguajes modernos, violaciones de precondiciones suelen manejarse con excepciones, aunque Pascal carece de este mecanismo.

Además de las operaciones básicas, es común extender el TAD Pila con funcionalidades adicionales que, sin ser estrictamente necesarias, aumentan su utilidad práctica:

- **`isFull(p)`**: Determina si la pila ha alcanzado su capacidad máxima (en implementaciones con arrays de tamaño fijo).
- **`clear(p)`**: Vacía completamente la pila, dejándola en el mismo estado que después de `initialize`.
- **`size(p)`** o **`count(p)`**: Devuelve el número de elementos actualmente en la pila.
- **`npop(p, n)`**: Elimina `n` elementos de la cima de una sola vez (útil en ciertos algoritmos).

Estas operaciones pueden implementarse en términos de las operaciones básicas o, para mayor eficiencia, mediante acceso directo a la representación interna cuando se justifique.

2. Implementación de una pila mediante un array

La implementación de una pila utilizando un array como almacenamiento subyacente es conceptualmente sencilla y eficiente en términos de velocidad de acceso. Para ello necesitamos dos componentes fundamentales: un array de tamaño fijo para almacenar los elementos y un índice entero (típicamente llamado `cima` o `top`) que indica la posición del elemento superior en el array. . El tipo de los elementos del array corresponderá al tipo base de la pila, y en cuanto al índice, lo habitual es que cuando la pila está vacía, este índice tenga el valor 0.

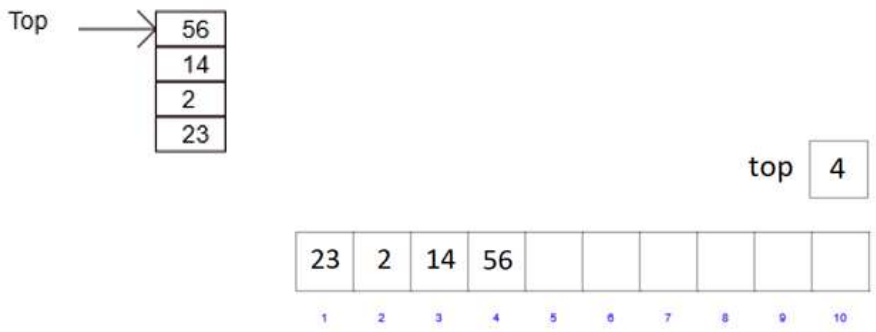

Ilustración 8. Implementación de un TAD pila mediante arrays

Tomaremos una serie de decisiones de diseño importantes, como por ejemplo definir una constante `MAX_ELEMENTOS` que establece la capacidad máxima de la pila. Además, al "eliminar" un elemento

(pop), no borraremos físicamente el valor del array; simplemente decrementamos el índice cima. El valor antiguo permanece en memoria pero se considera no válido.

En cuanto a la complejidad, esta implementación hace que las operaciones push y pop tengan complejidad temporal O(1) (tiempo constante), por lo que son muy eficientes.

Si utilizamos como tipo base los enteros, la implementación en Pascal sería algo así:

```
unit uPilaEnteros;

interface

const
  MAX_ELEMENTOS = 100;
type
  tPilaEnteros = record
    cima: integer;
    stack: array[1..MAX_ELEMENTOS] of integer;
  end;
procedure initialize(var p: tPilaEnteros);
procedure push(var p: tPilaEnteros; x: integer);
procedure pop(var p: tPilaEnteros);
function peek(p: tPilaEnteros): integer;
function isEmpty(p: tPilaEnteros): boolean;
function isFull(p: tPilaEnteros): boolean;  { Operación extendida }
procedure clear(var p: tPilaEnteros);       { Operación extendida }

implementation

procedure initialize(var p: tPilaEnteros);
begin
  p.cima := 0;  { Pila vacía }
end;

function isEmpty(p: tPilaEnteros): boolean;
begin
  isEmpty := (p.cima = 0);
end;

function isFull(p: tPilaEnteros): boolean;
begin
  isFull := (p.cima = MAX_ELEMENTOS);
end;

procedure push(var p: tPilaEnteros; x: integer);
begin
  if not isFull(p) then
  begin
    p.cima := p.cima + 1;
    p.stack[p.cima] := x;
  end
  else
    { Manejo de error: pila llena (overflow) }
    writeln('Error: Pila llena - no se puede hacer push');
end;

procedure pop(var p: tPilaEnteros);
begin
  if not isEmpty(p) then
    p.cima := p.cima - 1
  else
    writeln('Error: Pila vacía - no se puede hacer pop');
end;

function peek(p: tPilaEnteros): integer;
```

```
begin
  if not isEmpty(p) then
    peek := p.stack[p.cima]
  else
  begin
    writeln('Error: Pila vacía - no se puede hacer peek');
    peek := 0;  { Valor por defecto en caso de error }
  end;
end;

procedure clear(var p: tPilaEnteros);
begin
  p.cima := 0;  { Vacía la pila eficientemente }
end;

end.
```

La implementación con arrays presenta ventajas e inconvenientes claros. En cuanto a las ventajas, podemos citar las siguientes:

- **Simplicidad:** El código es fácil de entender y depurar.
- **Eficiencia:** Acceso directo a memoria contigua, buen comportamiento de caché.
- **Predictibilidad:** Uso de memoria conocido y constante.

Por otra parte, tiene un buen número de desventajas:

- **Tamaño fijo:** Se debe conocer o estimar un tamaño máximo `MAX_ELEMENTOS` en tiempo de compilación. Si se subestima `MAX_ELEMENTOS`, la pila se llenará; si se sobreestima, se desperdicia memoria.
- **Gestión de errores rudimentaria:** Pascal no tiene excepciones, por lo que debemos manejar los errores (overflow/underflow) de manera rudimentaria y poco robusta poniendo mensajes por consola.
- **Nota sobre precondiciones:** En la implementación mostrada, verificamos las precondiciones (`isFull` antes de `push`, `isEmpty` antes de `pop`/`peek`) e imprimimos mensajes de error. En un contexto real, podríamos utilizar códigos de error, variables globales de estado, o (en lenguajes que lo soporten) lanzar excepciones.

3. Implementación con memoria dinámica

La implementación dinámica de una pila utiliza una estructura de **nodos enlazados**, donde cada nodo contiene el dato (del tipo base de la pila) y un puntero al nodo anterior (el que está "debajo" en la pila). La pila en sí se representa mediante un puntero al nodo de la cima (o `nil` si la pila está vacía). Esta representación elimina la limitación de tamaño fijo: la pila puede crecer hasta agotar la memoria disponible del sistema.

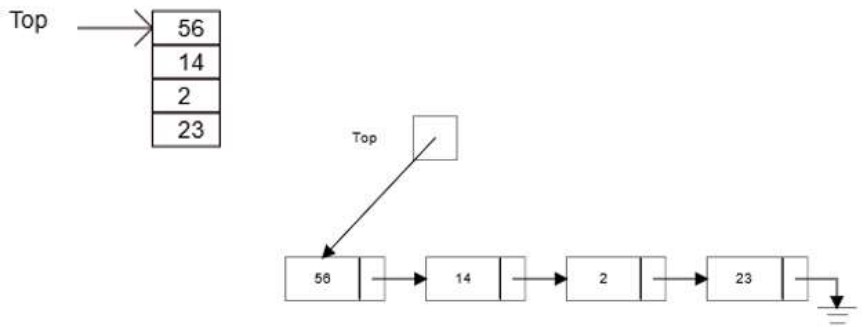

Ilustración 9. Implementación de un TAD pila mediante memoria dinámica

Las características clave de esta implementación son que no hay límite máximo predefinido (excepto la memoria disponible). Cada operación `push` requiere asignación dinámica (`new`) y cada operación `pop` requiere liberación de memoria (`dispose`). Las operaciones siguen siendo O(1) en complejidad temporal. Con respecto a la implementación mediante arrays, aquí es necesaria más memoria por elemento, pues necesita espacio para el puntero, pero es algo poco relevante.

Su implementación en Pascal sería algo así:

```
unit uPilaEnterosDinamica;

interface

type
  { Tipo nodo: representa un elemento de la pila }
  tNodo = record
    info: integer;
    anterior: ^tNodo;  { Puntero al nodo que está "debajo" }
  end;

  { La pila es un puntero al nodo de la cima }
  tPilaEnteros = ^tNodo;

procedure initialize(var p: tPilaEnteros);
procedure push(var p: tPilaEnteros; x: integer);
procedure pop(var p: tPilaEnteros);
function peek(p: tPilaEnteros): integer;
function isEmpty(p: tPilaEnteros): boolean;

implementation

procedure initialize(var p: tPilaEnteros);
begin
  p := nil;  { Pila vacía }
end;

function isEmpty(p: tPilaEnteros): boolean;
begin
  isEmpty := (p = nil);
end;

procedure push(var p: tPilaEnteros; x: integer);
var
  nuevo: ^tNodo;
begin
  new(nuevo);
  nuevo^.info := x;
  nuevo^.anterior := p;
  p := nuevo;
end;

procedure pop(var p: tPilaEnteros);
var
  aux: ^tNodo;
begin
  if not isEmpty(p) then
  begin
    aux := p;
    p := p^.anterior;
    dispose(aux);
  end
  else
    writeln('Error: Pila vacía - no se puede hacer pop');
end;

function peek(p: tPilaEnteros): integer;
```

```
begin
  if not isEmpty(p) then
    peek := p^.info
  else
  begin
    writeln('Error: Pila vacía - no se puede hacer peek');
    peek := 0;  { Valor por defecto }
  end;
end;

end.
```

4. Aplicaciones prácticas

Las pilas encuentran aplicación en numerosos dominios de la computación, demostrando que una estructura simple puede resolver problemas complejos cuando se aplica adecuadamente.

El uso más fundamental de las pilas en los sistemas computacionales es la pila de ejecución (*call stack*). Cada vez que se llama a una función o procedimiento se "apila" un marco de activación (*activation record*) que contiene: parámetros de la función, variables locales, dirección de retorno (dónde continuar después de la llamada) y otros datos de control. Cuando la función termina, su marco se "desapila" y se restaura el contexto de la función llamadora. Este mecanismo permite la recursividad (una función que se llama a sí misma) y gestiona automáticamente el ciclo de vida de las variables locales.

Las pilas son esenciales para evaluar expresiones aritméticas, especialmente en notación postfija (también llamada notación polaca inversa). El algoritmo es sencillo:

1. Recorrer la expresión de izquierda a derecha.
2. Si encontramos un operando (número), lo apilamos.
3. Si encontramos un operador (+, -, *, /), desapilamos los dos operandos superiores, aplicamos la operación, y apilamos el resultado.
4. Al final, el resultado estará en la cima de la pila.

Ejemplo: Evaluar “ `3 4 + 5 *` ” (equivalente a `(3+4)*5`)

Paso 1: Apilar 3 → Pila: [3]
Paso 2: Apilar 4 → Pila: [3, 4]
Paso 3: Encontrar `+` → Desapilar 4 y 3, calcular 3+4=7, apilar 7 → Pila: [7]
Paso 4: Apilar 5 → Pila: [7, 5]
Paso 5: Encontrar `*` → Desapilar 5 y 7, calcular 7*5=35, apilar 35 → Pila: [35]
Resultado: 35

Otra aplicación, que resulta un problema clásico resuelto con pilas, es verificar si una secuencia de símbolos de agrupación (paréntesis, corchetes, llaves) está correctamente balanceada. El algoritmo sería el siguiente:

1. Recorrer la expresión de izquierda a derecha.
2. Si encontramos un símbolo de apertura (`(`, `[`, `{`), lo apilamos.
3. Si encontramos un símbolo de cierre (`)`, `]`, `}`):
4. Si la pila está vacía → Error (cierre sin apertura)
5. Desapilamos y verificamos que coincida con el símbolo de apertura
6. Al final, si la pila no está vacía → Error (apertura sin cierre)

Este algoritmo se extiende naturalmente a la validación de documentos XML/HTML (balanceo de etiquetas) y a la verificación de sintaxis en lenguajes de programación, por mostrar dos ejemplos del mundo real muy utilizados.

Otras aplicaciones serían los navegadores web (pila adelante/atrás) o los editores de texto (pila deshacer/rehacer) utilizan pilas para gestionar el historial de acciones, así, en el navegador al navegar a una nueva página, se apila la anterior: "Atrás" desapila, "Adelante" requiere una segunda pila. En el caso del editor, cada acción de edición se apila: "Deshacer" desapila la última acción, "Rehacer" la vuelve a apilar en una pila secundaria.

En problemas de búsqueda como laberintos, el problema de las 8 reinas, o recorridos de grafos, las pilas permiten implementar la técnica conocida como *backtracking* (vuelta atrás) de manera natural. Cada decisión se "apila", y si llegamos a un callejón sin salida, "desapilamos" para probar alternativas. En un laberinto, por ejemplo, en cada intersección apilamos las direcciones no exploradas y cuando llegamos a un callejón sin salida, desapilamos para volver a la última intersección con alternativas pendientes.

Resumen

La pila, a pesar de su simplicidad conceptual, es una de las estructuras de datos más importantes en la computación. Su implementación directa (con arrays o listas enlazadas) sirve como excelente introducción a conceptos más avanzados de estructuras de datos, mientras que sus aplicaciones prácticas demuestran la utilidad de los principios abstractos estudiados en capítulos anteriores.

La pila sigue un principio LIFO: Último en entrar, primero en salir - simple pero potente. Sus operaciones básicas son: `push`, `pop`, `peek`, `isEmpty`, `initialize` - contrato mínimo.

Hay dos implementaciones principales: Con arrays, una implementación sencilla, eficiente, pero con limitaciones derivadas de su tamaño fijo; o con memoria dinámica, más flexible, con tamaño ilimitado pero también con una mayor necesidad de memoria por elemento.

En cuanto a su complejidad, todas las operaciones básicas son O(1) en ambas implementaciones.

Es una estructura que tiene un buen número de aplicaciones ubicuas: Call stack, evaluación de expresiones, validación de sintaxis, backtracking, historial de un navegador, etc.

El estudio de las pilas no solo nos proporciona una herramienta práctica, sino que también refuerza los conceptos fundamentales de la asignatura: la importancia de una especificación clara (TAD), la separación entre interfaz e implementación (abstracción), y el análisis de eficiencia (complejidad algorítmica). En los próximos capítulos, estudiaremos otras estructuras lineales (colas, listas) y no lineales (árboles, grafos), cada una con sus propias características y aplicaciones. Sin embargo, muchos de los principios aprendidos aquí -especialmente el uso de memoria dinámica y el diseño de interfaces limpias- se aplicarán de manera similar en esas estructuras más complejas. La pila es, en muchos sentidos, la puerta de entrada al mundo rico y diverso de las estructuras de datos.